Ravan Mehraliyev

Principles of Operating Systems (CSCI-3510-20319)

Assignment 3

In this assignment, I implemented a file system allocation simulator in Python. The main goal of this assignment was to simulate how an operating system stores files on disk and keeps track of which disk blocks belong to which file. I implemented three file allocation methods which are contiguous allocation, FAT allocation, and i-node allocation. I also added a hierarchical directory system, hard links, soft links, and a simple journal log for delete operations. My purpose in this assignment was not to build a real operating system, but to simulate the main ideas in a clear way and compare the advantages and disadvantages of each allocation method.

How Directory System and Links Work

I implemented a hierarchical directory system so that files can be created under folders like /docs. I also implemented both hard links and soft links.

Hard Link: It is another entry that points to the same file. Because of that, deleting the original file name does not remove the file data immediately if another hard link still exists. In my test, I deleted /docs/report.txt, but /docs/report_hard.txt still worked correctly. This shows that the reference count idea was handled properly.

Soft Link: This one is different from a hard link. It behaves like a separate file that stores the target path. If the original target file is deleted, then the soft link becomes broken. In this assignment, after deleting the original file, the hard link remained accessible, while the soft link became invalid. This is the correct behavior.

How Journaling Works

I used a basic journal log for deleting operations. The simulator logs the steps to be followed before deleting a file. As an illustration, the delete operation involves such steps as deleting the directory record, freeing the file metadata, and reallocating disk blocks to free space. This is a basic simulation of journaling systems such as NTFS or ext3. My implementation is straightforward yet it demonstrates the overall concept that the system records what it intends to do, then it does it.

Workload Used for Testing

I tried the simulator with the same workload of each of the three allocation methods in order to make the comparison fair. The workload was: create directory /docs, create file /docs/report.txt, open the file, write 4 blocks, read 2 blocks, close the file, create a hard link, create a soft link, delete the original file, open the hard link, read 2 blocks again and close the file. I performed the same workload on contiguous allocation, FAT allocation and i-node allocation.

Results After Execution

I used the same workload for all three allocation methods. The summaries below are based on the actual runs after execution.

Method	Key Result	Meaning
Contiguous	Used blocks = 4 free blocks = 28 external fragmentation = 0.0 contiguous read seeks = 1	The file operations worked correctly, and the remaining free space ended up as one large free extent. This workload showed correctness more than fragmentation.
FAT	Used blocks = 4 free blocks = 28 FAT memory overhead = 128 bytes, chain traversals = 14	The file also worked correctly here, but FAT needed extra in-memory overhead for the table. This clearly showed the memory cost of FAT.
I-node	Used blocks = 4 free blocks = 28 loaded inodes = 0 inode memory overhead = 0 bytes at end	At the end of the workload, all files were closed, so no inode remained loaded in memory. This showed that i-nodes are only kept in memory when needed.

All three methods worked correctly. They were able to create files, open and close files, write and read blocks, delete files, support directories, and support hard and soft links. Hard links and soft links behaved differently as expected. The hard link still worked after deleting the original file name, while the soft link became broken because its target path no longer existed.

Firstly, Contiguous allocation was simple and fast for reading. It had the smallest read seek count in my test. On the other hand, FAT had extra memory overhead because the FAT table had to stay in memory. In my result, it was 128 bytes. Last one, I-node allocation handled memory better because metadata only needed to be in memory when files were open.

Screenshots of the Execution

Contiguous

```
PS C:\Users\Ravan\Desktop\OS_Assigment3> python main.py --strategy contiguous --workload workload_example.txt
Workload execution log:
 - MKDIR /docs
 - CREATE /docs/report.txt
 - OPEN /docs/report.txt -> fd 3
 - WRITE fd 3 4
 - READ fd 3 2 -> []
 - CLOSE fd 3
 - LINK_HARD /docs/report.txt /docs/report_hard.txt
 - LINK_SOFT /docs/report.txt /docs/report_soft.txt
 - DELETE /docs/report.txt
 - OPEN /docs/report_hard.txt -> fd 4
 - READ fd 4 2 -> [0, 1]
 - CLOSE fd 4

Summary:
{
  "strategy": "contiguous",
  "disk_blocks": 32,
  "free_blocks": 28,
  "used_blocks": 4,
  "journal_entries": 1,
  "strategy_stats": {
    "free_extents": 1,
    "largest_free_extent": 28,
    "external_fragmentation": 0.0
  },
  "contiguous_read_seeks": 1
}
```

Fat

```
PS C:\Users\Ravan\Desktop\OS_Assigment3> python main.py --strategy fat --workload workload_example.txt
Workload execution log:
 - MKDIR /docs
 - CREATE /docs/report.txt
 - OPEN /docs/report.txt -> fd 3
 - WRITE fd 3 4
 - READ fd 3 2 -> []
 - CLOSE fd 3
 - LINK_HARD /docs/report.txt /docs/report_hard.txt
 - LINK_SOFT /docs/report.txt /docs/report_soft.txt
 - DELETE /docs/report.txt
 - OPEN /docs/report_hard.txt -> fd 4
 - READ fd 4 2 -> [0, 1]
 - CLOSE fd 4

Summary:
{
  "strategy": "fat",
  "disk_blocks": 32,
  "free_blocks": 28,
  "used_blocks": 4,
  "journal_entries": 1,
  "strategy_stats": {
    "fat_memory_overhead_bytes": 128
  },
  "fat_chain_traversals": 14
}
```

I-Node

```
PS C:\Users\Ravan\Desktop\OS_Assigment3> python main.py --strategy inode --workload workload_example.txt
Workload execution log:
 - MKDIR /docs
 - CREATE /docs/report.txt
 - OPEN /docs/report.txt -> fd 3
 - WRITE fd 3 4
 - READ fd 3 2 -> []
 - CLOSE fd 3
 - LINK_HARD /docs/report.txt /docs/report_hard.txt
 - LINK_SOFT /docs/report.txt /docs/report_soft.txt
 - DELETE /docs/report.txt
 - OPEN /docs/report_hard.txt -> fd 4
 - READ fd 4 2 -> [0, 1]
 - CLOSE fd 4

Summary:
{
  "strategy": "inode",
  "disk_blocks": 32,
  "free_blocks": 28,
  "used_blocks": 4,
  "journal_entries": 1,
  "strategy_stats": {
    "loaded_inodes": 0,
    "inode_memory_overhead_bytes": 0
  }
}
```

Advantages and Disadvantages of Each Method

Method	Advantage	Disadvantage
Contiguous	simple to implement, fast reading, easy to track block range.	can suffer from external fragmentation, harder to grow files if the next blocks are not free.
FAT	files do not need contiguous blocks, easier file growth, no external fragmentation in the same way as contiguous allocation.	FAT table must remain in memory, access may require following many pointers.
I-node	file metadata is separate and organized, only needed i-nodes are loaded in memory, supports direct and indirect pointers.	more complex to implement, slightly more metadata management is needed.

Conclusion

In this assignment, I built a file system allocation simulator that supports contiguous allocation, FAT allocation, and i-node allocation. I also implemented a directory system, hard links, soft links, and a basic journaling mechanism.

From my testing, I can say that all three allocation methods worked correctly for the workload I used. The hard link remained valid after deleting the original file, while the soft link became broken, which is the expected result. Contiguous allocation showed good read efficiency, FAT showed memory overhead because of the in-memory table, and i-node allocation showed that metadata only needs to stay in memory when files are open.